
LAN Multi-User Communication System

Real-time collaboration platform

with video conferencing, audio, screen sharing,
chat, and AI assistance

Contributors

Yadynesh (CS23B1055)

Pavan (CS23B1069)

Table of Contents

1. System Overview

2. Installation & Setup

3. Communication Protocols

4. User Guide

5. AI Assistant

6. Troubleshooting

7. Advanced Configuration

1 System Overview

The LAN Multi-User Communication System enables real-time collaboration on local networks through video conferencing, audio streaming, screen sharing, group chat, and file transfer. The system operates entirely within local networks without external dependencies, prioritizing privacy, low latency, organizational infrastructure control, and cross-platform compatibility.

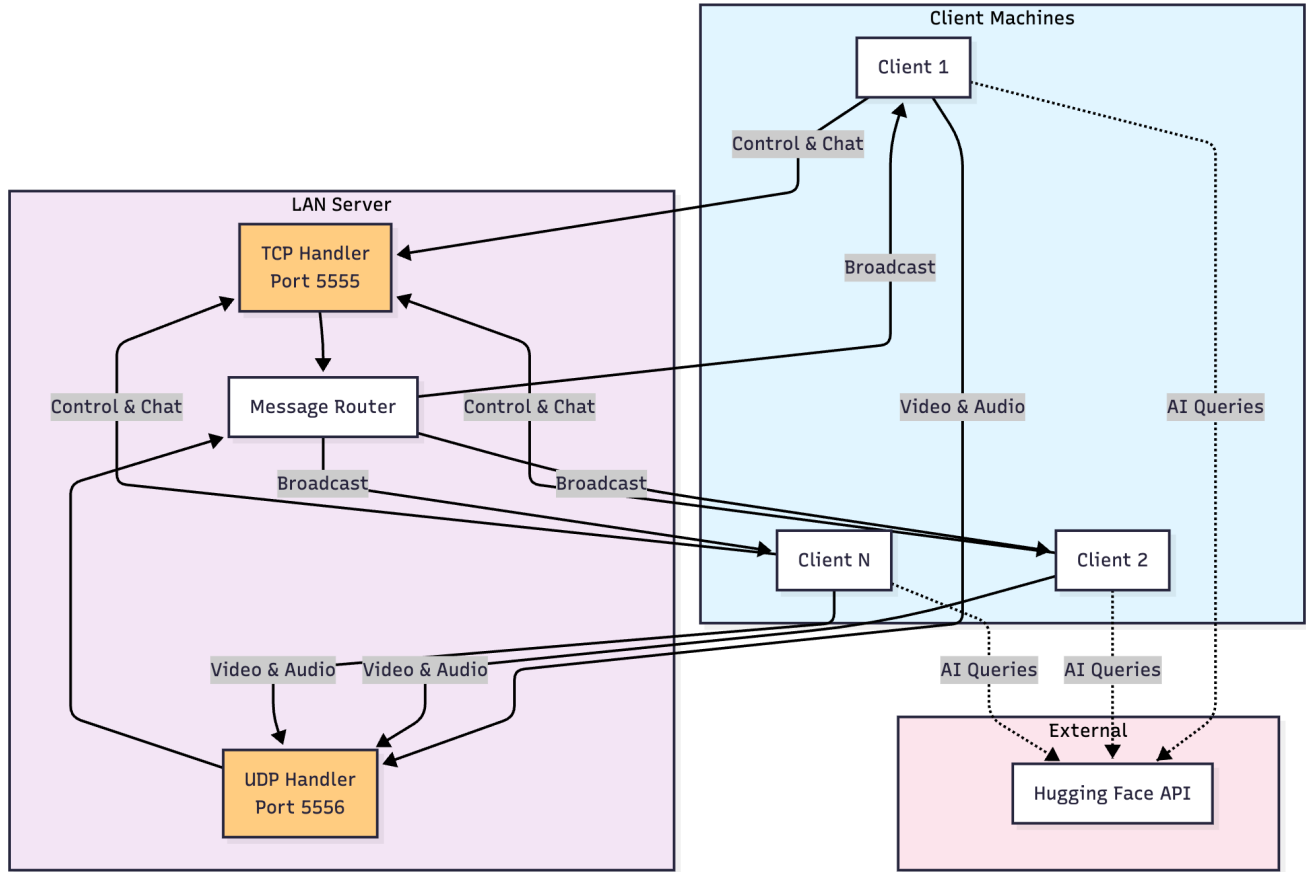


Figure 1: **System Architecture Overview:** A centralized LAN server supports multiple clients, using TCP port 5555 for control messages and chat, and UDP port 5556 for real-time video/audio streaming. The server includes TCP Handler, UDP Handler, and Message Router to manage sessions and broadcast media. An external Hugging Face API provides AI features like intelligent assistance and chat summarization.

1.1 Key Features

- **Video Conferencing:** Multi-user video streaming at 320×240 resolution, 30 FPS with adaptive quality compression
- **Audio Communication:** Full-duplex voice communication at 16 kHz mono PCM format
- **Screen Sharing:** Live desktop presentation at 10 FPS with 800×600 resolution
- **Group Chat:** Persistent messages with automatic timestamps, full history for new users
- **File Sharing:** Upload and download between participants with base64 encoding for TCP transmission
- **AI Assistant:** Query Llama-3.2 via Hugging Face API with chat summarization capabilities
- **User Management:** Automatic session tracking and real-time user list updates

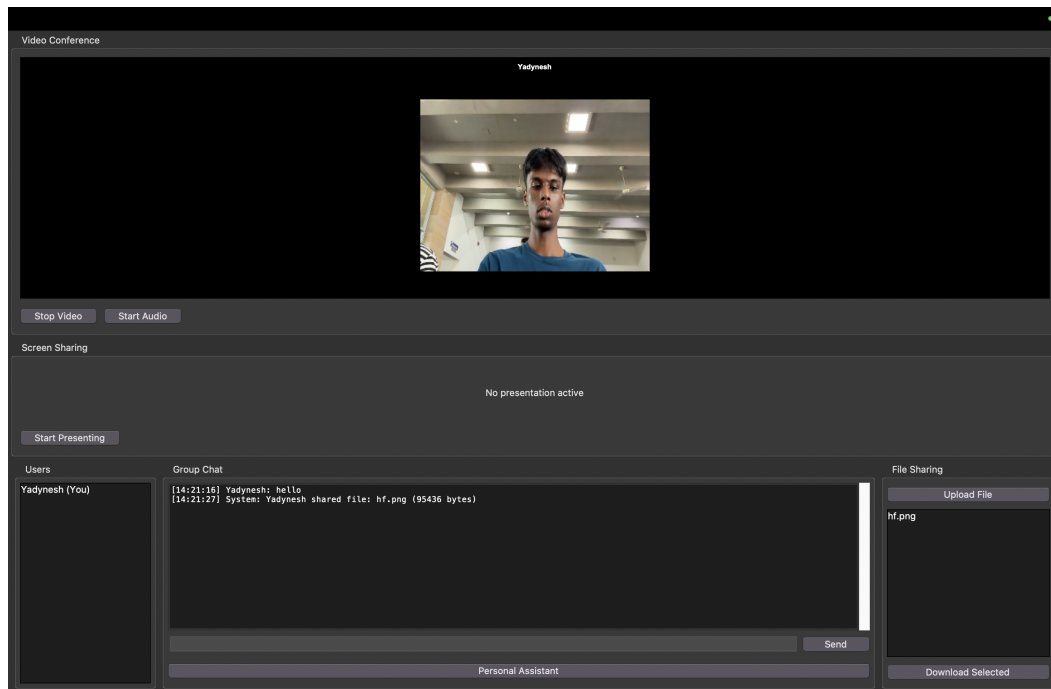


Figure 2: **Main Interface:** A multi-user video conferencing interface with a three-panel layout: left panel lists connected users with online status, center displays real-time video streams in a grid, and right panel contains audio/video controls. Supports simultaneous participants with automatic stream management.

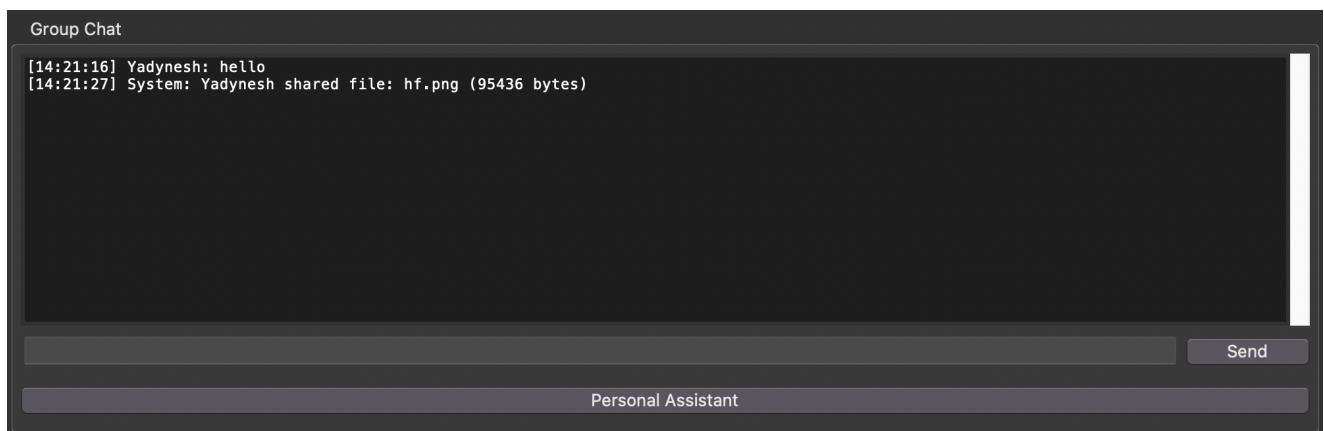


Figure 3: **Chat Panel:** A persistent group chat interface showing all messages with HH:MM:SS timestamps, sender usernames, and content. System messages indicate user join/leave events, and new participants receive the complete chat history on connecting.

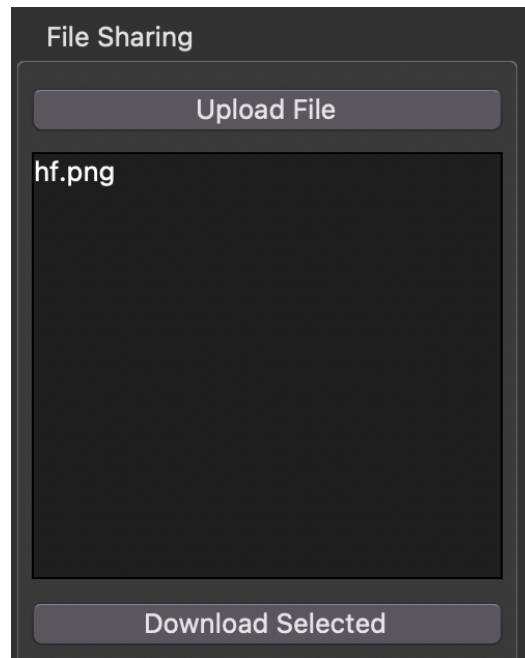


Figure 4: **File Transfer Interface:** File sharing feature allowing participants to upload and download files. Displays available files with names and sizes, with options to upload from the user's computer or download selected files shared by others.

1.2 System Architecture

The system follows a client-server architecture combining TCP for reliable control messages with UDP for efficient real-time streaming. The client side uses tkinter for GUI, OpenCV for video processing, PyAudio for audio streaming, and MSS for screen capture. The server side manages TCP connections on port 5555 for control messages and UDP on port 5556 for media streaming. TCP handles chat messages, file transfers, and control signals while UDP handles real-time video and audio streams.

1.3 System Requirements

- Python 3.8 or higher
- Local Area Network (LAN) with all devices on same network
- Ports 5555/5556 accessible through firewall
- Webcam, microphone, and speakers
- 2GB RAM minimum
- Dual-core CPU or better
- 1200×800 display minimum
- Minimum 2 Mbps bandwidth per video participant
- Wired Ethernet recommended for server

2 Installation & Setup

2.1 Step 1: Install Dependencies

Download the system and install required packages:

```
cd lan-communication-system
pip install -r requirements.txt
```

Required packages include opencv-python, numpy, Pillow, pyaudio, mss, and requests.

2.2 Step 2: Get Server IP

Identify the server machine's IP address on your local network.

Linux/macOS:

```
hostname -I
```

Windows:

```
ipconfig
```

Note your server IP address (for example, 192.168.1.100).

2.3 Step 3: Start Server

On the designated server machine:

```
python3 server.py
```

The server will display output showing the server IP and listening ports:

```
=====
LAN Communication Server
=====
Server IP: 192.168.1.100
TCP Port: 5555
UDP Port: 5556
=====
```

2.4 Step 4: Start Clients

On each client machine, run the client application:

```
python3 client.py
```

A connection dialog will appear. Fill in the following information:

- **Username:** Your name or identifier
- **Server IP:** The IP address from Step 2
- **TCP Port:** 5555
- **UDP Port:** 5556
- Click **Connect** to join the session

3 User Guide

3.1 Interface Layout

The client application presents a comprehensive interface divided into several sections:

Top: Video Conference Area

Grid of all active video streams, auto-arranged to maximum 3 columns. Start Video and Start Audio buttons for stream control. Username label displayed on each video stream. Video displays at 320×240 resolution with adaptive quality.

Middle: Screen Sharing Area

Start Presenting and Stop Presenting buttons. Only one presenter allowed at a time; other presenter buttons become disabled. Screen content updates at 10 FPS with 800×600 resolution. 60% quality compression for bandwidth efficiency.

Bottom: Users, Chat, and Files Section

Left panel contains connected users list with online status. Center panel shows chat input field and message history with timestamps. Right panel provides file upload/download interface.

3.2 Using Video Conferencing

To start video streaming, click the Start Video button and grant camera permission when prompted. Your video will appear in the grid along with other participants' videos as they join. Video streams automatically arrange in columns. To stop streaming, click the Stop Video button.

3.3 Using Audio Communication

Click the Start Audio button and grant microphone permission. Audio then broadcasts to all participants. Use headphones to prevent echo and feedback. Audio operates continuously at 16 kHz, 256 kbps. To stop audio, click the Stop Audio button.

3.4 Using Screen Sharing

Click the Start Presenting button to capture and stream your entire desktop at 10 FPS. Only one person can present at a time; other participants' presenter buttons become disabled. When you finish, click Stop Presenting to end the screen share.

3.5 Using Group Chat

Type your message in the input field at the bottom and press Enter or click Send. Messages broadcast immediately with automatic timestamps in HH:MM:SS format. All messages persist; new users receive the full chat history upon joining. System messages announce when users join or leave.

3.6 Using File Transfer

To upload a file, click the Upload File button and select a file from your computer. All users receive a notification. To download, select a file from the file list and click Download Selected, then choose your save location. The system supports all file types with a practical limit of 100MB per file.

4 Communication Protocols

4.1 TCP Protocol for Reliable Messaging

TCP handles registration, chat, file transfer, and presentation control using length-prefixed JSON messages. The protocol format includes a 4-byte length prefix followed by the JSON payload. This ensures reliable delivery through acknowledgments and retransmission.

TCP Message Types:

Registration messages allow clients to connect and establish their identity in the system. Chat messages include the username, message content, and timestamp. File upload messages contain the filename, file size, and base64-encoded file data.

4.2 UDP Protocol for Real-Time Streams

UDP carries video and audio with minimal overhead for low-latency transmission. The packet structure includes a type byte (1 for video, 2 for audio), a username length byte, the username, and the payload.

Video Specifications:

- Codec: JPEG compression
- Resolution: 320×240 pixels
- Frame Rate: 30 FPS
- Quality: 40% compression (adaptive)
- Bandwidth: Approximately 600 kbps
- Maximum packet size: 65,507 bytes

Audio Specifications:

- Format: PCM 16-bit mono
- Sample Rate: 16 kHz
- Chunk Size: 2,048 samples (128ms duration)
- Bandwidth: 256 kbps

Screen Sharing Specifications:

- Resolution: 800×600 pixels
- Frame Rate: 10 FPS
- Quality Compression: 60%
- Bandwidth: 500–800 kbps

The system tolerates up to 5% UDP packet loss without significant quality degradation.

5 AI Assistant

The Personal Assistant integrates the Hugging Face Inference API with the Llama-3.2-3B-Instruct language model to provide intelligent responses and chat analysis.

5.1 Setting Up AI Assistant

To enable AI features:

1. Create an account at <https://huggingface.co>
2. Log in to your account and navigate to Settings
3. Click Access Tokens and then New token
4. Name the token (for example, “LAN-Communication”)

5. Select Read scope
6. Generate and copy the token
7. Edit client.py around line 45
8. Replace YOUR_TOKEN_HERE with your actual token in the line: `HF_HEADERS = {"Authorization": "Bearer hf_..."}`
9. Restart the client application to activate

5.2 Using the AI Assistant

Click the Personal Assistant button in the chat panel to open the AI dialog window.

Chat with AI Feature:

Ask questions and receive instant answers. Type your query and press Enter. AI responds within 3–10 seconds. Maximum response length is 300 tokens. Example queries include “What is machine learning?”, “Explain this concept”, or “How do I use this app?”

Summarize Group Chat Feature:

Automatically summarizes all group messages. Highlights key discussions and interactions. Click the Summarize Chats button. AI analyzes the chat history and generates a comprehensive summary. Identifies key topics, decisions, and participant interactions.

5.3 AI Integration Details

The system queries the Llama-3.2-3B-Instruct model via the Hugging Face router API endpoint at <https://router.huggingface.co/v1/chat/completions>. Request parameters include a system prompt stating “You are a helpful assistant”, the user message, a maximum of 300 tokens for response length, and temperature set to 0.0 for deterministic responses. Typical response time ranges from 3–10 seconds. On timeout or network issues, an error message displays suggesting to check your internet connection or verify token validity.

5.4 Troubleshooting AI Assistant

“HF request failed: 401” error indicates an invalid or missing API token. Solution: Generate a new token and update the HF_HEADERS in client.py.

Slow responses exceeding 30 seconds suggest API server slowness or network latency. Solution: Check your internet connection and verify the Hugging Face API status.

6 Troubleshooting

6.1 Connection Issues

“Connection refused” error means the server is not running or the IP address and port are incorrect. Verify the server has been started on the correct machine, check the IP address with the ping command, confirm ports 5555/5556 are accessible, and check that your firewall allows these ports.

“Connection timeout” error suggests network issues. Ping the server to verify connectivity, ensure all devices are on the same LAN, and test your WiFi or Ethernet connection stability.

6.2 Video Issues

“Cannot access webcam” error requires granting camera permission in your operating system settings, closing other applications using the camera, or reconnecting the USB device.

Black screen with no video can be resolved by trying a different USB port for the camera, updating camera drivers, or restarting the application.

Choppy or stuttering video improves by using a wired network connection instead of WiFi, closing heavy background applications, moving closer to your router, reducing the number of participants, or restarting the application.

6.3 Audio Issues

“No audio devices” error can be checked by verifying the device count with Python or reinstalling PyAudio. Test command:

```
python3 -c "import pyaudio; print(pyaudio.PyAudio().get_device_count())"
```

“Device in use” error indicates another application is using the audio device. Close applications like Zoom, Teams, or other conference software.

No sound from other participants requires checking your speaker volume and mute status, testing speakers with other applications, verifying other users are streaming, and restarting the application.

Echo or feedback is best solved by using headphones instead of speakers, reducing microphone gain, or moving your microphone away from speakers.

6.4 Chat and File Issues

Messages not appearing suggests a connection problem. Check your connection status and reconnect if necessary.

File upload fails may indicate a file size issue or unstable connection. Try uploading a smaller file and ensure a stable network connection.

Corrupted file downloads can be resolved by downloading the file again.

7 Advanced Configuration

7.1 Custom Server Ports

Start the server with custom ports if the default ports are unavailable:

```
python3 server.py 8888 8889
```

This launches the server with TCP on port 8888 and UDP on port 8889.

7.2 Small Organizations (5–20 users)

Use a single server on a dedicated machine with wired Ethernet for the server. WiFi is acceptable for client connections. Ensure network has minimum 2 Mbps bandwidth per participant.

7.3 Medium Organizations (20–100 users)

Deploy a dedicated network switch with QoS (Quality of Service) configuration. Implement network monitoring to track performance and bandwidth usage.

7.4 Large Deployments (100+ users)

Multiple servers with load balancing are required. Implement SSL/TLS encryption for security. Use a database backend for persistent chat and file storage. Deploy professional network monitoring and management systems.

7.5 Performance Optimization

For low bandwidth scenarios, disable video and use audio-only mode, close background applications, use wired network connections, and reduce display resolution. For low-power devices, close unnecessary applications, reduce video resolution, and avoid multitasking.

Start with a small number of users for network testing, monitor quality metrics, and adjust resolution and frame rates as needed. Perform regular maintenance including weekly log reviews, monthly dependency updates, periodic backup verification, and continuous network monitoring.

7.6 Chat History Persistence

The system automatically persists chat messages. All messages are stored and new users receive the complete history upon joining. For backup purposes, chat history is retained throughout the session.

8 Conclusion

The LAN Multi-User Communication System provides a production-ready collaboration platform combining reliable TCP control with efficient UDP streaming. Key achievements include adaptive video conferencing, full-duplex audio communication, desktop screen sharing, persistent group chat with AI summarization, secure file transfer, and cross-platform compatibility.

Benefits include complete privacy with all data remaining local, low latency suitable for interactive use, full organizational infrastructure control, independence from cloud services, and complete customization capability. The system scales from 5-person teams to 100+ user deployments through configuration. Supported technologies include Python 3, OpenCV for video, PyAudio for audio, Tkinter for GUI, Pillow for image processing, MSS for screen capture, and Hugging Face API for AI features.

This system is suitable for small and medium organizations, educational institutions, research facilities, and government environments prioritizing data security and infrastructure autonomy.

Happy communicating!